

CHITALA OS

Blueprint 2026-2046 · Version 20

Một hệ điều hành độc lập cho thế giới AI, robot, thiết bị vật lý và compute phân tán

Nguyên tắc trung tâm v20

Chitala có thể chạy trên Linux, Windows, macOS, Android, iOS hoặc RTOS trong giai đoạn bootstrap, nhưng kiến trúc Chitala không phụ thuộc vào bất kỳ hệ điều hành chủ, CPU ISA, AI runtime, giao thức hay cloud provider cụ thể nào. Đích dài hạn là Chitala Native: hệ thống có thể boot và vận hành trực tiếp trên phần cứng.

v20 giữ tham vọng của các blueprint trước về một OS mới, đồng thời tách rõ Core Architecture, Hosted Runtime, Native Runtime, Fabric và Future Profiles để tránh biến dự án thành một tập hợp tính năng không kiểm soát.

Trạng thái: Architecture blueprint / research & implementation roadmap. 03/10/2026.

1. Tuyên ngôn kiến trúc

Chitala không được định nghĩa đơn thuần là middleware chạy trên Linux. Linux là nền tảng bootstrap thực dụng, không phải nền móng bắt buộc của kiến trúc.

Đối tượng trung tâm của Chitala không chỉ là process, file và user. Chitala xem Human, AI, Agent, Robot, Device, Service và Compute Resource là các thực thể có identity, capability, authority, state và provenance.

Chuỗi kiểm soát nền tảng:

Principal → Identity → Capability → Intent → Authority → Reference Monitor → Execution → State → Audit

- Không ambient authority cho AI/agent; quyền phải rõ nguồn gốc, giới hạn và thu hồi được.
- Không hành động vật lý nếu thiếu bằng chứng ủy quyền và điều kiện an toàn.
- Protocol và data model phải language-neutral; Rust là reference implementation, không phải hợp đồng vĩnh viễn.
- Core phải nhỏ, kiểm chứng được; integrations và profiles nằm ngoài Trusted Computing Base khi có thể.
- Kiến trúc phải chịu được thay đổi phần cứng và mô hình AI trong ít nhất hai thập niên.

2. Chitala là OS theo nghĩa nào?

Lớp	Vai trò
Chitala Core	Identity, capability, policy, reference monitor, secure messaging, state, audit, resource/intent model.
Chitala Runtime	Lifecycle, services, eventing, scheduling/execution abstraction, local IPC.
Chitala System API	Hợp đồng ổn định giữa ứng dụng/agent và hệ thống.
Platform Abstraction Layer	Tách clock, storage, IPC, entropy, key store, network, process/thread, device I/O khỏi host OS.
Native substrate	Kernel/microkernel/hypervisor + HAL/drivers khi Chitala chạy độc lập.
Fabric	Mở rộng identity/authority/state/execution qua nhiều node.

Vì vậy Chitala có thể tồn tại trước khi có kernel riêng, nhưng thiết kế phải cho phép thay host OS bằng native substrate mà không viết lại Trusted Core.

3. Hai chế độ vận hành bắt buộc

3.1 Hosted Mode - xây dựng ngay

Chitala Runtime chạy như một hệ thống dịch vụ trên Linux trước tiên, sau đó có backend Windows/macOS khi cần. Android/iOS chủ yếu tham gia như edge/client node.

Apps / Agents → Chitala API → Trusted Core → PAL → Linux/Windows/macOS → Hardware

Mục tiêu Hosted Mode: phát triển nhanh, kiểm thử security model, xây simulator, driver/adapters và chứng minh product-market fit mà chưa phải gánh chi phí kernel/drivers.

3.2 Native Mode - đích độc lập

Chitala Native phải có khả năng boot trực tiếp trên phần cứng mà không cần Linux, Windows, macOS, Android hay iOS.

Apps / AI / Agents
↓
Chitala Runtime & Services
↓
Trusted Core / System API
↓
Native PAL / HAL
↓
Kernel hoặc microkernel/hypervisor substrate
↓
CPU · Memory · Storage · Network · GPU/NPU · Devices

v20 không bắt buộc tự viết kernel từ ngày đầu. Quyết định dùng kernel tự phát triển, microkernel có sẵn hay hypervisor substrate là quyết định implementation về sau, miễn System API và security invariants không bị phụ thuộc vào lựa chọn đó.

4. Platform Abstraction Layer (PAL)

PAL là thay đổi kiến trúc cần làm sớm nhất trong code hiện tại để Chitala không bị khóa vào Unix/Linux.

Abstraction	Không được hard-code
TimeSource	system clock Unix cụ thể; phải hỗ trợ monotonic/secure time.
SecureKeyStore	file key; backend có thể là TPM, Secure Element, enclave hoặc software.
IpcTransport	Unix socket; có thể thay bằng named pipe, shared memory, native IPC.
Storage	POSIX path semantics.
Entropy	một OS RNG cụ thể.
NetworkTransport	TCP/MQTT/HTTP cụ thể; transport là plugin.
ExecutionHost	process/thread model của Linux.
Devicelo	sysfs/devfs cụ thể; HAL/driver capability.

5. Trusted Core v20

Trusted Core nên tiếp tục kế thừa phần code hiện có: model, identity, capability token, policy, secure message envelope, reference monitor, audit, state và event bus. Tuy nhiên dependency hướng xuống phải đi qua PAL.

Khối	Bất biến chính
Identity	Mọi principal có định danh mật mã; hỗ trợ rotation, ownership epoch và attestation về sau.
Capability	Quyền có scope, expiry, delegation depth; delegation chỉ thu hẹp.
Intent	Biểu diễn mục tiêu tách khỏi lệnh thiết bị; planner không tự có authority.
Authority	Tổng hợp identity + token + policy + context + safety + human approval.
Reference Monitor	Một đường kiểm soát bắt buộc; fail closed.
CSME	Envelope có chữ ký, anti-replay, version negotiation, canonical encoding.
State/Twin	Reported/desired/observed state; provenance và anti-rollback.
Audit	Tamper-evident; checkpoint/anchor ngoài node khi cần.
Safety	Invariant cục bộ gắn actuator; cloud/AI không được bypass.

6. Principal model

Chitala mở rộng khái niệm principal để phản ánh thế giới hậu-PC.

Principal	Ví dụ
Human	owner, clinician, operator, technician
AI/Agent	LLM agent, planner, vision agent, autonomous workflow
Robot	humanoid, AMR, drone, surgical/industrial robot
Device	sensor, lock, infusion device, actuator
Service	database, broker, cloud service, local daemon
Compute	CPU pool, GPU/NPU accelerator, remote compute node
Organization/Domain	home, hospital, factory, vehicle fleet

AI là principal hạng nhất nhưng không mặc nhiên là trusted principal. Khả năng suy luận và authority là hai khái niệm tách biệt.

7. Resource Model - không khóa vào phần cứng 2026

Compute được biểu diễn bằng capability và thuộc tính, không bằng tên CPU/GPU cụ thể.

```
ComputeResource ::= CPU | GPU | NPU | DSP | FPGA | RemoteCompute | Neuromorphic | QuantumAccelerator | FutureCompute
```

Scheduler/execution planner có thể chọn tài nguyên dựa trên latency, energy, locality, confidentiality, determinism, cost và safety class. Một workload không nên biết nó đang chạy trên ISA nào trừ khi thật sự cần.

Tương tự, storage, network và actuator đều được mô tả qua interface/capability ổn định thay vì API vendor.

8. Intent & Execution Model

Một OS cho AI không nên để natural-language prompt trở thành system call. Prompt/goal phải được chuyển thành Intent có schema, sau đó mới lập kế hoạch và xin authority.

```
Human/Agent Goal → Intent → Plan → Capability Resolution → Policy/Safety Check → Execution Lease → Action → Observation → Audit
```

- Planner có thể thay thế: symbolic, LLM, rule engine, optimizer hoặc future AI.
- Plan không tạo quyền. Mỗi action phải chứng minh authority độc lập.
- Execution lease có deadline, resource budget, scope và cancellation semantics.
- High-risk action có thể yêu cầu Human Decision Center; no-response = deny.
- Physical action phải có local safety invariant ở node/actuator.

9. AI Runtime Independence

Chitala không đồng nhất AI với LLM và không phụ thuộc MCP như giao thức lỗi. MCP chỉ là một adapter.

```
IntelligencePrincipal ::= LLM | VisionModel | Planner | Agent | RobotIntelligence | Human | FutureIntelligence
```

AI provider, model format và inference runtime phải nằm sau adapter. Một domain có thể đồng thời dùng local NPU, remote model, deterministic planner và human operator mà security model không thay đổi.

10. Chitala Fabric

Fabric là sự mở rộng của cùng security model qua nhiều node, không phải một cloud bắt buộc.



- Identity và capability xuyên node nhưng policy có thể local-first.
- Node mất mạng vẫn duy trì safety island cho các chức năng thiết yếu.
- Discovery không đồng nghĩa trust; enrollment và attestation tách biệt.
- State replication phải có provenance, conflict semantics và anti-rollback.
- Federation giữa domain dùng explicit trust contract; không có global superuser.

11. Device, Robot và Physical World

Đây là miền tạo khác biệt lớn cho Chitala. Device Action không phải side effect phụ của app; nó là một primitive có risk, precondition, postcondition, rollback/recovery và audit.

Thuộc tính action	Ví dụ
Capability	door.unlock, pump.set_rate, robot.move
Risk class	low / medium / high / safety-critical
Safety envelope	range, rate limit, geofence, collision constraints
Precondition	door closed, patient verified, area clear
Authority	human/token/policy/human approval
Evidence	sensor state, attestation, timestamp
Recovery	stop, safe state, compensate, escalate

12. Security Architecture

- Zero ambient authority; least privilege bằng capability.
- Reference Monitor là mandatory mediation point.
- Keys dần chuyển từ file sang hardware-backed non-exportable credentials.
- Secure/monotonic time cho expiry và anti-rollback.
- Audit checkpoint ra ngoài node; transparency log tùy profile.
- Adapter và parser chạy ngoài Trusted Core khi có thể.
- Memory-safe implementation cho core; unsafe/FFI phải cô lập và audit.
- Supply-chain security: pinned dependencies, SBOM, provenance, signed releases, reproducible builds.
- Coverage-guided fuzzing cho wire formats, IPC, token, adapters.
- Post-quantum agility: crypto suite negotiation; không hard-code một thuật toán vĩnh viễn.

13. Kernel strategy

v20 không tuyên bố phải tự viết kernel ngay. Có ba con đường có thể thử nghiệm song song:

Chiến lược	Ưu điểm	Nhược điểm
Hosted	Nhanh nhất, driver ecosystem sẵn có	Không độc lập; host compromise ảnh hưởng Chitala
Microkernel/native substrate	TCB nhỏ, isolation tốt, hợp capability model	Driver/tooling và engineering cost cao
Hypervisor-first	Cô lập legacy OS/workloads; migration thực dụng	Phức tạp virtualization, không phù hợp mọi MCU

Quyết định kernel chỉ nên được khóa sau khi System API, PAL, threat model và native prototype chứng minh được yêu cầu thực tế.

14. Chitala Tiny

MCU không cần chạy full Chitala. Chitala Tiny cung cấp identity, secure envelope, minimal capability enforcement, state/event subset và local safety.

Chitala Tiny → HAL/RTOS hoặc bare metal → ESP32 / STM32 / RISC-V MCU / future MCU

Profile Tiny phải có giới hạn bộ nhớ, message size, crypto suite và offline behavior rõ ràng. Nó vẫn là participant của Fabric nhưng không giả vờ có đầy đủ chức năng server.

15. Compatibility & Legacy

Chitala không yêu cầu thế giới bỏ Linux/Windows/Android/iOS. Legacy systems được bao quanh bởi adapters/gateways và được cấp quyền như untrusted/partially trusted principals.

- Linux/Windows/macOS: host backend và legacy workload.
- Android/iOS: edge/client node, identity holder, human approval surface.
- RTOS: substrate cho Chitala Tiny hoặc adapter.
- MQTT/HTTP/WoT/Home Assistant: protocol adapters.
- MCP và các agent protocol tương lai: AI adapters.
- POSIX/WASI/container/VM: compatibility profiles, không phải Chitala core contract.

16. Profiles - giữ tầm nhìn nhưng không làm phình Core

Profile	Phạm vi
Home/Building	IoT, energy, access, automation.
Medical	device/patient identity, safety-critical action, provenance.
Robot/Humanoid	motion/action capability, local safety, human override.
Mobility	vehicle/drone/eVTOL authority, geofence, fleet federation.
Industrial	PLC/robot/process control, deterministic safety islands.
Compute	distributed CPU/GPU/NPU pools, confidential execution, marketplace.
Personal	vault, personal agents, device continuity.
Future Profiles	Digital Immune System, advanced federation, emerging hardware.

Profile không được kéo dependency ngược vào Core. Core chỉ bổ sung primitive khi có ít nhất hai miền độc lập cần cùng một abstraction.

17. Roadmap 2026-2046

Giai đoạn	Mục tiêu
2026-2027 · Core	Ổn định spec; PAL; CI/SBOM/fuzz; intent/resource model; adapters sandbox; simulator.
2027-2029 · Host	Linux production node; Windows/macOS backend khi cần; Home/Robot/Medical experiments; multi-node fabric.
2029-2032 · Native Lab	Bootable Chitala Native prototype; microkernel/hypervisor evaluation; minimal drivers; secure key/time.
2032-2036 · Native	Native node cho edge/server/robot; compatibility VM/container; robust update/recovery.
2036-2040 · Fabric	Cross-domain federation, heterogeneous compute, large-scale provenance, safety islands.
2040-2046 · Evolution	Crypto migration, new compute paradigms, new intelligence forms, compatibility without architectural reset.

18. Việc cần thay đổi trong repository hiện tại

- Tạo crate `chitala-platform` định nghĩa PAL traits: time, entropy, key store, storage, IPC, network, execution host.
- Di chuyển Unix socket, POSIX permissions/path và system clock ra khỏi Trusted Core.
- Giữ `chitala-mcp`, Home Assistant và MQTT/WoT ở adapter layer; không cho chúng định nghĩa core semantics.
- Thêm `chitala-resource` cho hardware/compute/resource descriptors.
- Thêm `chitala-intent` cho typed intent, plan và execution lease; không cho prompt đi thẳng tới device action.
- Chuẩn hóa version negotiation và crypto agility cho CSME/wire formats.
- CI: fmt, clippy, test, audit, deny; SBOM, signed release, reproducible build.
- Fuzz CSME/token/IPC/adapters và chaos/mixed-version tests.

- Tách adapter process khỏi node/monitor; giới hạn capability IPC.
- Viết Native Architecture ADR nhưng chưa triển khai kernel trước khi PAL ổn định.

19. Các điều Chitala v20 cố ý KHÔNG khóa

- Không khóa vào Linux hay POSIX.
- Không khóa vào x86, ARM hay RISC-V.
- Không khóa vào GPU/NPU vendor.
- Không khóa vào LLM, Transformer hay một AI provider.
- Không khóa vào MCP, MQTT, HTTP hoặc một transport.
- Không khóa vào cloud.
- Không khóa vào một database/filesystem.
- Không khóa vào một crypto algorithm duy nhất.
- Không khóa vào kernel strategy trước khi có bằng chứng kỹ thuật.
- Không biến Future Profiles thành dependency của Core.

20. Tiêu chí để gọi Chitala là OS độc lập

Chitala đạt mốc Native OS khi một hệ thống tham chiếu có thể:

- Boot từ firmware/bootloader vào Chitala mà không cần Linux/Windows/macOS/Android/iOS.
- Khởi tạo memory, timer, interrupt, storage/network tối thiểu qua native substrate/HAL.
- Khởi động Trusted Core và system services.
- Chạy ứng dụng/agent qua Chitala System API.
- Enforce identity/capability/policy/reference monitor cho local và remote action.
- Quản lý resource/execution và lifecycle.
- Update/recover an toàn, audit được và fail closed.
- Kết nối Chitala Fabric và vẫn giữ local safety khi mạng/cloud mất.

Trước mốc này, Hosted Chitala vẫn là implementation hợp lệ của kiến trúc, nhưng không được tuyên bố rằng host OS là thành phần bắt buộc của Chitala.

21. Nguyên tắc kiểm soát độ phức tạp

Tầm nhìn rộng không đồng nghĩa triển khai đồng thời mọi thứ. Mỗi primitive mới vào Core phải vượt qua ba câu hỏi: (1) có phải abstraction lâu dài không; (2) có ít nhất hai profile cần nó không; (3) nếu bỏ nó, Chitala có mất một thuộc tính OS cốt lõi không?

Mobility, eVTOL, Humanoid, Compute Marketplace và Digital Immune System được giữ trong tầm nhìn 2046, nhưng phát triển theo profile/research track. Điều này giữ sự khác biệt của Chitala mà không tạo second-system effect.

22. Definition of Done cho v0.1 mới

- PAL tồn tại và Trusted Core không import API Unix trực tiếp ngoài backend.
- 94+ test hiện tại tiếp tục pass; thêm PAL contract tests.
- CI/security pipeline hoạt động.
- Coverage-guided fuzz targets cho CSME/token/IPC.

- Intent v0.1 và Resource Model v0.1 có spec + implementation tối thiểu.
- Adapter isolation prototype.
- Linux hosted node hoạt động như trước.
- Native architecture ADR + boot experiment tối thiểu, nhưng chưa cần full kernel.
- Threat model cập nhật cho hosted vs native trust boundary.

23. Tuyên bố v20

Chitala là một kiến trúc hệ điều hành hướng tới thế giới mà con người, AI, robot, thiết bị và compute phân tán cùng chia sẻ tài nguyên và tác động lên thế giới vật lý. Chitala dùng identity, capability, intent, authority, safety, state và provenance làm các primitive cốt lõi.

Chitala có thể được triển khai trên hệ điều hành hiện hữu để phát triển và tương thích, nhưng không phụ thuộc kiến trúc vào chúng. Đích dài hạn là Chitala Native và Chitala Fabric, trong đó cùng một security model có thể trải từ MCU đến robot, edge, workstation và datacenter.

Mục tiêu 2026–2046 không phải dự đoán chính xác phần cứng hay AI tương lai. Mục tiêu là xây những abstraction đủ ổn định để Chitala có thể hấp thụ các thay đổi đó mà không phải viết lại nền tảng.